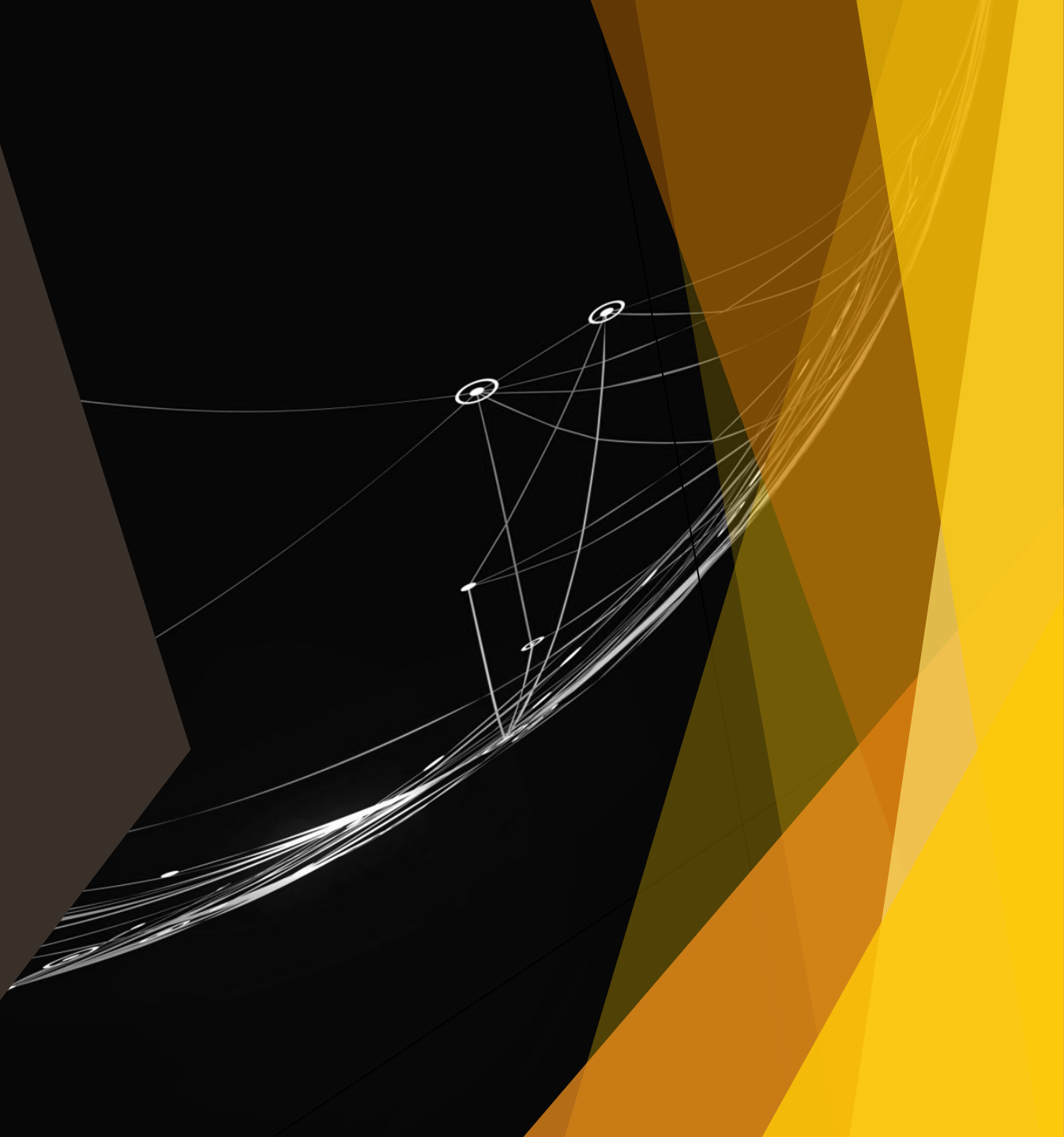
Tema 5. Sobrecarga, conversiones, errores y más

a) Sobrecarga de operadores
b) Relaciones de amistad (friends)
c) Plantillas (*Templates*) y programación genérica
d) Conversiones seguras (castings)
e) Gestión de errores y excepciones



Sobrecarga de Operadores

- Las siguientes operaciones están definidas tanto para tipos primitivos como para clases:
- Declaración/construcción normal y por copia, Asignación por copia:

```
int a = 2;
Complejo c1 = Complejo(3.4, 2.5);
```

```
int b = a;
Complejo c2 = c1;
```

```
int c;
c = b;
Complejo c3;
c3 = c2;
```

- ¿Pero, es posible hacer lo siguiente?

```
a+b;
c1 + c2;
```

- Ya que fue posible sobrecargar el **operador** de asignación por copia, también se puede pensar que es posible sobrecargar casi todas las **operaciones** con las que solemos **operar:** suma +, resta -, multiplicación *, división /, comparación <, flujo de salida << , flujo de entrada >>
- Todos estos **operadores** pueden ser definidos en las clases que creemos.

Sobrecarga de Operadores

- Reestructurando una definición previa de la clase Complejo, tenemos el siguiente diagrama al que podemos agregar un nuevo método que sobrecargue al operador de suma + utilizando la palabra reservada operator seguido del operador a sobrecargar:

Complejo
- real: float - imag: float
+ Complejo() + Complejo(Complejo &otro) + getReal(): float + getImag(): float + setReal(float real): void + setImag(float imag): void + toString(): string + operator=(Complejo &otro): Complejo **+ operator+(Complejo &otro): Complejo**

- En el fichero de encabezado, declaramos el método:

```
Complejo operator+(const Complejo& otro);
```

- Mientras que en el .cpp, implementamos la operación:

```
1  Complejo Complejo::operator+(const Complejo& otro) {
2      Complejo resultado;
3      resultado.real = real + otro.real;
4      resultado.imag = imag + otro.imag;
5      return resultado;
6  }
```

Sobrecarga de Operadores Aritméticos

>01

- Todos los operadores pueden ser definidos, incluyendo:
  Aritméticos + - * / % = += *= /=
  Relación < > >= == <=
- La diferencia clave entre los operadores **aritméticos** es que los que lleven el operador de asignación (=) modifican y devuelven el objeto original, mientras los otros no (crean y devuelven un 3er objeto).

Tema6 - Complejo.cpp

```cpp
Complejo Complejo::operator+(const Complejo& otro) {
    Complejo resultado;
    resultado.real = real + otro.real;
    resultado.imag = imag + otro.imag;
    return resultado;
}
```

Tema6 - Complejo.cpp

```cpp
Complejo& Complejo::operator+=(const Complejo& otro)
{
    real += otro.real;
    imag += otro.imag;
    return *this;
}
```

Sobrecarga de Operadores Aritméticos

>02

- Algo interesante es que no es obligatorio que el elemento con el cual se opera sea de la misma clase.
- Por ejemplo, se puede sobrecargar operadores para trabajar con flotantes
- + operator*=(const float & num): Complejo&
- + operator+(const float & num): Complejo

Tema6 - Complejo.cpp

```
Complejo& Complejo::operator*= (const float& num){
    real *= num;
    imag *= num;
    return *this;
}
```

Tema6 - Complejo.cpp

```
Complejo Complejo::operator+(const float& num){
    Complejo resultado;
    resultado.real = real + num;
    resultado.imag = imag;
    return resultado;
}
```

Sobrecarga de Operadores Lógicos y de Relación

- También pueden ser sobrecargados los operadores lógicos && || !
- Y de relación > < == <= >=
- Es fácil notar que en este último caso los resultados esperados de estos operadores NO son objetos sino un valor **bool**: true o false.
- La sobrecarga de estos operadores solo debe usarse cuando el resultado tiene un significado real tanto en la vida real como en programación.
- Ejemplo: Al definir una Clase Matriz, el operador de == si puede definirse ya que dos matrices iguales tienen los mismos datos, pero los operadores < y > carecen de sentido por lo que no deberían definirse.

Sobrecarga de Operadores Lógicos y de Relación

- + operator==(const Complejo & otro): Complejo&

Tema6 - Complejo.h

```
bool operator==(const Complejo& otro);
```

Tema6 - Complejo.cpp

```
bool Complejo::operator==(const Complejo& otro) {
    return (real == otro.real) && (imag == otro.imag);
}
```

En el main:

```
c1 == c2
```

Sobrecarga de Operadores

- Como sabemos, la sobrecarga de métodos no está definida para un prototipo en particular sino para el mismo nombre con diferentes parámetros.
- Por esta razón es posible seguir sobrecargando operadores para conseguir que una clase pueda operar con otra clase o tipo primitivo.
- En el caso de Complejo sería muy útil sobrecargar el operador = para inicializar objetos de la clase Complejo con el parámetro real definido, es decir que en el main podamos utilizar:

```
1  Complejo c;
2  c = 5; // 5 no es un Complejo, ni un float ni un double, es un int
```

- Por lo tanto, con la definición correcta, nuestro .cpp podría incluir:

```
1  Complejo& Complejo::operator=(const int& real){
2      this->real = real;
3      this->imag = 0;
4      return *this;
5  }
```

Sobrecarga de Constructores

- Este método también es extrapolable a los constructores.
- En caso de los números complejos podemos probar la diferencia entre el llamado a una construcción por copia y una asignación por copia si definimos diferentes comportamientos en estas funciones.
- Para ellos podríamos, por ejemplo, definir que en construcción copiamos un flotante al número real y en el operador al número imaginario.

Tema6 - Complejo.h

```
// Asignación por copia de un int
Complejo& operator=(const int& real);
// Constructor por copia de un int
Complejo(const int& imag);
```

Tema6 - Complejo.cpp

```
Complejo& Complejo::operator=(const int& real){
    this->real = real;
    this->imag = 0;
    return *this;
}

Complejo::Complejo(const int& imag){
    this->real = 0;
    this->imag = imag;
}
```

```
c = 5;
Complejo d = 5;
```

Sobrecargas (Method Overloading)

- Desarrolla en C++ una clase Vector3D que represente un vector geométrico en el espacio tridimensional (con coordenadas x, y, z de tipo float).
- Deberás implementar lo siguiente:
- **Constructores:** Un constructor por defecto (inicializa en 0,0,0) y un constructor parametrizado que recibe un puntero a flotante.
- **Asignación (=):** Sobrecarga el operador para permitir copiar los valores de un vector a otro de forma segura.
- **Suma (+):** Sobrecarga el operador aritmético para devolver un nuevo Vector3D que sea el resultado de sumar las coordenadas de dos vectores.
- **Comparación (==):** Sobrecarga el operador relacional para evaluar si dos vectores tienen exactamente las mismas coordenadas.
- **Indexación ([]):** Sobrecarga este operador para que devuelva el elemento del índice seleccionado.
- **Requisito final:** Escribe una función main que instancie múltiples objetos de tu clase y ejecute pruebas para demostrar que cada uno de los constructores y operadores funciona.

Más relaciones entre clases

- La sobrecarga de operadores y constructores puede funcionar para realizar operaciones y construcciones entre diferentes clases, es decir no solamente entre Complejo e int sino también entre Complejo y una clase nueva llamada Vector2D.

Tema6 - Vector2D.h

```
Vector2D(const Complejo& comp);
```

Tema6 - Vector2D.cpp

```
Vector2D::Vector2D(const Complejo& comp) {
    x = comp.real;
    y = comp.imag;
}
```

- Pero como los atributos de Complejo son privados, no es posible compilar este código.
- Para solucionar dicho problema recurrimos a establecer una relación nueva entre clases: La relación de amistad, usando la palabra reservada `friend`

Relaciones de amistad

- friend es la palabra reservada que se utiliza para obtener acceso a funciones/clases privadas desde otra función/clase externa.
- Para ello, tenemos que definir el lazo de amistad en el diagrama de clases.
- Se utiliza la línea discontinua B - - - - > A para reflejar que la clase B es amiga de A (por lo que B puede acceder a los miembros/funciones privadas de A)
- La relación no es simétrica. Esto significa que, según el ejemplo mostrado, A no puede acceder a los métodos de B.
- Si ambas clases son amigas se usa A < - - - - - > B
  (esto requiere que se defina la relación de amistad en ambas clases)

Relaciones de amistad

- La relación de amistad se especifica dentro del apartado público en la clase que comparte sus datos.
- Para nuestro primer ejemplo, en el header de Complejo (clase que comparte datos) habrá que definir que Vector2D es su amiga (Vector2D es amiga de Complejo), con el siguiente formato:

Tema6 - Complejo.h

```
1  friend class Vector2D;
```

- Con esto Vector2D tiene acceso a todos los elementos privados de Complejo.

Relaciones de amistad

- Para usar Vector2D deberíamos incluir "Vector2D.h", pero como esta clase depende de Complejo, y Complejo depende de Vector2D, entonces creamos un bucle infinito de dependencias.
- Para solucionar este inconveniente, procedemos a declarar la existencia de la clase Vector2D en Complejo.

Tema5 - Complejo.h

```
class Vector2D;
```

- Por lo que para declarar amistad debemos:
  1. Reconocer la clase que debe compartir los datos.
  2. En la cabecera de dicha clase, declarar la existencia de la otra clase.
  3. Dentro de la clase, declarar la relación de amistad.

Relaciones de amistad

- Una forma de recordar la relación es que la cabeza de la flecha siempre apunta a la clase de donde se obtienen datos.
- Recordar que en Herencia la sub-clase obtiene los datos de la super-clase, por lo que la cabeza de la flecha también está en esa dirección.
- Por fin, con la relación de amistad entre clases, podemos observar que el siguiente código se ejecuta sin errores debimos a la declaración hecha en la clase que “es amiga de”

```
Complejo c = Complejo(3.4, 2.5);
Vector2D v = c;
```

Relaciones de amistad

>06

- Existen situaciones en las que queremos que una función cualquiera que no forma parte de ninguna clase acceda a los elementos privados de una clase.
- Por suerte, `friend` se puede utilizar tanto para clases como para funciones, por lo que será posible crear una función graficaModulo(Vector2D v) en cualquier parte de nuestro programa:

Tema6 - Vector2D.h

```
1  friend void graficaModulo(Vector2D v);
```

```
1  void graficaModulo(Vector2D v)
2  {
3      float modulo = sqrt(v.x * v.x + v.y * v.y);
4      string salida = "|";
5      int magnitud_en_pantalla = (int)round(modulo * 30);
6      for (int i = 0; i < magnitud_en_pantalla; i++)
7      {
8          salida += "-";
9      }
10     salida += ">\n\n";
11     cout << salida;
12 }
```

Relaciones de amistad

- La función amiga más definida en C++ tiene la siguiente firma:

Tema6 - Vector2D.h

```
friend ostream & operator<<(ostream &os, Vector2D& v);
```

- La clase ostream se encuentra definida dentro de iostream y es esta la que se define como amiga de la clase Vector2D. En cualquier parte de la memoria estática debe definirse la función:

```
ostream & operator<<(ostream &os, Vector2D& v){
    return os << "Vector: (" << v.x << ", " << v.y << ")";
}
```

- Desde ahora en adelante, en el main se puede utilizar lo siguiente:

```
Vector2D v = Vector2D(3.4, 2.5);
cout << v << endl;
```

Relaciones de amistad

- **Objetivo:** Implementar un sistema de acceso a datos utilizando **relaciones de amistad** para vulnerar la encapsulación de una clase protegida.
- Tu objetivo es interactuar con la clase ServidorCentral. Esta clase tiene medidas de seguridad extremas: sus atributos son estrictamente private y no existen métodos públicos para acceder a ellos.
- **La Clase ServidorCentral :**
  - **Atributos privados:** string codigo(ej: "admin:root123") y int nivel_seguridad(un valor de 1 a 10).
  - **Constructores:** para inicializar los datos.
  - **Amistad:** Debe declarar explícitamente como "amigos" a una función de packetSniffing, a la clase TestHacker y a los operadores de flujo >> y <<.
- **Amistad con Funciones:**
  - Implementa una función global llamada packetSniffing(const ServidorCentral& s).
  - Esta función debe interceptar el tráfico y mostrar por consola el contenido de codigo accediendo directamente al miembro privado.
- **Amistad entre Clases:**
  - Crea la clase TestHacker.
  - Debe contener un método modificarCodigo(ServidorCentral& s, string codigo) que sobrescriba la base de datos del servidor.
- **Sobrecarga de Operadores de Flujo :**
  - **Operador de Salida (<<):** Al enviar el servidor a cout, debe imprimir un log (ej: [SYSTEM LOG] Firewall: [X/10] | Data: [ENC]).
  - **Operador de Entrada (>>):** Debe permitir al usuario definir el nivel_seguridad del servidor desde la consola para cambiarlo manualmente.
- **Ejecución en main:**
  - Instancia un ServidorCentral con el firewall al máximo.
  - Ejecuta el packetSniffing para revelar los codigos.
  - Utiliza >> para disminuir el nivel_seguridad a 0.
  - Usa un objeto de TestHacker y modificar el código.
  - Muestra el estado final del servidor usando <<.

Programacion Avanzada >08

- Podríamos crear la función

```
bool comparaModulo(Vector2D v, Complejo c);
```

- Y deberíamos de asignar a comparaModulo como una función amiga tanto de Vector2D como Complejo.

```
friend bool comparaModulo(Vector2D v, Complejo c);
```

- Y si existiese una tercera clase llamada Vector3D y una cuarta clase ElementoDeEspacioVectorial para comparar los módulos entre estas 4 clases deberíamos de escribir 4C2 = 10 funciones y para una 5ta clase adicional cuyo módulo pueda ser obtenido necesitaríamos 15 …….
- ¿No sería mejor escribir una sola vez dicha función?

Plantillas: Templates

- Una plantilla es una función en donde existen elementos intercambiables de cualquier tipo, no solamente clases.
- Supongamos que deseemos encontrar el mínimo valor de un arreglo de ints.
- Para ello tendremos que crear una función que obtenga el arreglo y la cantidad de elementos:

```
1  int& encontrarMinimo(int * a, int n) {
2      int i_min = 0;
3       for (int i = 0; i < n - 1; i++)
4           if (*(a + i) < *(a + i_min))
5               i_min = i;
6       return *(a + i_min);
7   }
```

- Con esto fue posible encontrar el menor de los elementos de un arreglo de enteros … pero ¿qué pasaría que también necesitemos encontrar el menor de un arreglo de floats? ¿Y de doubles? ¿Y de Complejos?

Plantillas: Templates

- Lo que podemos hacer es convertir nuestra función de datos específicos a una plantilla que pueda recibir diferentes tipos de datos, pero que cumplan con el objetivo de la función de forma idéntica.
- Para ello, debemos indicar que la función es una plantilla `template` y que existen uno o más datos que son de ningún tipo especificado `<typename T>`.
- Luego, cada vez que, en ese ámbito utilicemos dicho nombre de variable, el compilador de C++ ajusta dicho tipo según el dato que se pase como argumento al llamar a la función.

```
1  template <typename T> T& encontrarMinimo(T * a, int n) {
2      int i_min = 0;
3       for (int i = 0; i < n - 1; i++)
4           if (*(a + i) < *(a + i_min))
5               i_min = i;
6       return *(a + i_min);
7  }
```

Plantillas: Templates

- Así, si el primer argumento corresponde a un arreglo de enteros entonces la función trata a los elementos como enteros. En el caso de flotantes y dobles, etc. sucede lo mismo.

```
int a[] = {1, 2, 3, 4, 5};
cout << "El minimo es: " << encontrarMinimo(a, 5) << endl;
float b[] = {1.1, 2.2, 3.3, 4.4, 5.5};
cout << "El minimo es: " << encontrarMinimo(b, 5) << endl;
```

- Todo esto habiendo definido solamente una función ya que la comparación de menor que está definida para los tipos primitivos.
- ¿Cómo se debe hacer para que se pueda obtener el mínimo de un arreglo de Vector2D?

Plantillas: Templates

- Siempre y cuando el operator< esté definido, se podrá utilizar la función plantilla.

Tema6 - Vector2D.cpp

```
1  bool Vector2D::operator<(const Vector2D& otro){
2      return (x*x + y*y) < (otro.x*otro.x + otro.y*otro.y);
3  }
```

- La palabra reservada **typename** puede ser opcionalmente modificada a **class** para especificar que la función plantilla se ha escrito exclusivamente para clases. En cualquier caso, el uso es el mismo:

```
1  Vector2D vectores[] = {Vector2D(1, 2), Vector2D(3, 4),
   Vector2D(5, 6)};
2  Vector2D v = encontrarMinimo(vectores, 3);
```

Plantillas: Templates

- Para implementar plantillas de forma sencilla hay que tener en cuenta 2 cosas importantes:
  - Se debe escribir la función plantilla de forma a que se puedan utilizar solamente operadores (es decir, no utilizar funciones o métodos genéricos de alguna clase)
  - Se debe escribir siempre la función dentro del main.cpp y no dentro de otros ficheros como utils.cpp
- Por supuesto, estas 2 normativas hacen que la implementación y utilización de las plantillas sea útil solamente en el programa principal.

Plantillas Avanzadas

- Volviendo al ejemplo de comparaModulo(), empecemos haciendo que los parámetros sean plantillas, de distintos tipos. Para más de una plantilla se escriben los tipos separados por comas:

```
template<class T, class U> bool comparaModulo(T v, U c){
    return c.modulo() == v.modulo();
}
```

- Por supuesto, es necesario que ambas clases implementen todas las funciones que se utilizan como plantillas, en este caso, es obligatorio que las clases implementen la función .modulo()

Tema6 - Complejo.cpp

```
float Complejo::modulo(){
    return sqrt(real*real + imag*imag);
}
```

Tema6 - Vector2D.cpp

```
float Vector2D::modulo(){
    return sqrt(x*x + y*y);
}
```

Plantillas Avanzadas

- Una forma sencilla de realizar plantillas es comenzar escribiendo como si los tipos fuesen conocidos, y priorizar el uso de operadores (lógicos, de relación y aritméticos) siempre que sea posible.
- Escribir una función que permita seleccionar el menor de dos elementos de cualquier tipo.
- Empezamos la función considerando, de ser posible, 2 enteros:

Tema6 - Clase6.cpp

```cpp
int seleccionaMenor(int a, int b){
    return a < b ? a : b;
}
```

Tema6 - Clase6.cpp

```cpp
template<class T> T seleccionaMenor(T a, T b){
    return a < b ? a : b;
}
```

- Se declara la existencia de un tipo plantilla y se modifican los tipos que sean de dicha plantilla. Note que el operador < existe solo para Vector2D.

Plantillas Avanzadas

- Para utilizar un template en una clase, se ha de definir el template justo delante de la definición de la clase:

Tema5 - Mochila.h

```
1  template <typename T> class Mochila {
2      private:
3          T * datos;
4          int n;
5          ...
```

- Luego ya podemos utilizar T libremente dentro del .h
- Pero como una plantilla se utiliza solo durante la compilación de código, y no durante ejecución, no podemos crear un .cpp de una plantilla genérica.
- Entonces, debemos usar la tercera forma: la clase debe estar completamente definida en el .h

Plantillas Avanzadas

- Finalmente, a la hora de instanciar un objeto de la clase, debemos asegurarnos de utilizar un objeto compatible durante la creación.
- En el main, ya queremos trabajar con un tipo especifico: float, int, Documento, Personaje, etc. Por lo que debemos utilizar la forma correcta de templates al instanciar el objeto, es decir usar el tipo correcto y no T.
- Para utilizar en el main ha de describirse el tipo

Tema5 - Tema5.cpp

```
Carpeta<int> c1 = Carpeta<int>(5);
Carpeta<Complejo> c2 = Carpeta<Complejo>(10);
Carpeta<Personaje> c2 = Carpeta<Personaje>(100);
```

Plantillas Avanzadas

- El resto de la operación es indiferente con lo que habíamos definido para c1 o para c2, por lo que no se necesita ni definir el tipo, ni usar corchetes angulares.

Tema5 - Tema5.cpp

```
for(int i = 0; i < 5; i++){
    c1.modificar(i, i);
}
cout << "El tercer dato es " << c1[2] << endl;
```

Plantillas

La FIA desea un sistema informático para modelar dos aspectos críticos de un Gran Premio:

1. **La Parrilla de Salida:** Una secuencia de pilotos donde cada uno ocupa una posición y "apunta" al competidor que tiene inmediatamente detrás.
2. **El Trazado del Circuito:** Una serie de puntos de control (Checkpoints) que los pilotos deben cruzar en un orden estricto.

Implementar en **C++** una estructura de **Grafo** basada en plantillas.

1. Estructuras de Datos

- **Clase Nodo<T>:** Debe contener el dato genérico y un único puntero al siguiente elemento.
- **Clase GrafoF1<T>:** Debe permitir la creación de una secuencia vinculada de nodos. Debe incluir métodos para:
  - insertar(T dato): Añadir un elemento al final de la secuencia.
  - imprimir(): Recorrer el grafo y mostrar la jerarquía

2. Main

- El programa debe instanciar dos grafos distintos:
- **Grafo de Pilotos:** Utilizando string, debe representar el orden de salida (Pole Position, P2, P3...).
- **Grafo de Pista:** Debe representar la distancia de cada segmento.

Restricciones Técnicas

- **Gestión de Memoria:** Es obligatorio implementar un destructor en la clase Grafo que libere la memoria de todos los nodos creados dinámicamente para evitar fugas (*memory leaks*).
- **Encapsulamiento:** Los atributos de las clases deben ser privados o protegidos, utilizando constructores para la inicialización.

Gestión de errores

- Existen varias clases de **errores** y **excepciones**, pero se pueden agrupar en 3 grandes clases: de compilación, de ejecución y lógicos.
- Los errores de compilación son errores que se presentan en tiempo de compilación, son errores que el compilador sabe que ocurrirán en caso de que el programa se ejecute. Por lo que evita la compilación del programa y “lanza” un error de compilación.
- Un programa final se pretende que sea un fichero compilado (.exe, ….) que, por supuesto, ya no tiene errores de compilación.
- Pero podría tener errores de ejecución que detengan el programa de forma inesperada, o errores lógicos, que eviten que el programa realice el comportamiento deseado.
- C++ permite la gestión de estos errores.

Gestión de errores

- Es casi imposible crear un programa complejo que no falle a la primera vez.
- Pero también resulta muy difícil crear un programa completo y genérico que pueda funcionar para todos los casos.
- Por ejemplo, una clase se puede definir para definir un complejo con: Complejo c = 5;
- Pero un programador no sabe que en nuestro caso no es posible construir: Complejo c = “5 + 0i”; // Desafío: crear un constructor con string
- Ya que todavía no definimos ese constructor, ocurriría un problema que no ocurre de forma frecuente. Esto es algo no generalizado porque, en condiciones normales, el constructor debería funcionar, entonces es una **excepción** del programa.

Gestión de errores de compilación

- Una **excepción** indica que ha ocurrido un problema cuando se ha ejecutado un programa.
- C++ está construido para manejar las excepciones lanzando advertencias o errores según el problema: compilación, o ejecución.
- Las excepciones se extienden más allá de las clases y objetos, por ejemplo, la instrucción int i = 5.3; lanza una excepción del tipo advertencia que imprime en pantalla **“main.cpp(26,8): warning C4244: 'inicializando': conversión de 'double' a 'int'; posible pérdida de datos”.**
- Este tipo de excepción es de tipo compilación porque únicamente ocurre cuando se compila el programa.
- Separemos este mensaje en cinco diferentes partes.

Gestión de errores de compilación

- main.cpp(26,8): ¿Dónde ha ocurrido la excepción?
- warning C4244: ¿De qué tipo es la excepción?
- 'inicializando': ¿En qué momento ha ocurrido?
- conversión de 'double' a 'int'; ¿Qué operación se ha realizado?
- posible pérdida de datos" ¿Cuál es el resultado?

- En resumen, estamos en presencia de una excepción del tipo advertencia, que, si bien permite que el programa continue, ha hecho alguna operación que no es esperada, o no se espera que funcione de forma correcta.
- **Otra cosa importante es que este tipo de excepciones es de compilación.**

Gestión de errores de compilación

- Realice la operación `Complejo c = "4+ 3i";`
- Compile el programa, por supuesto, que dará un error.
- Lea el mensaje, separe en las 5 partes como en el ejemplo anterior y reflexione sobre los resultados.

Gestión de errores de ejecución

- Durante la **ejecución** de un programa pueden producirse distintos tipos de errores, como errores de programación, entradas no válidas o situaciones inesperadas. Cuando se produce un error, C++ normalmente se detiene y genera un mensaje de error. El término técnico para esto es: C++ lanza una excepción (genera un error).
- El manejo de excepciones te permite detectar y gestionar los errores durante la ejecución, para que tu programa no se bloquee.
- Utiliza tres palabras clave:
  - try: define el código que se va a probar
  - throw: lanza una excepción
  - catch: gestiona el error
- Se usan principalmente para situaciones no gestionadas por código

```
1  try {
2    // Codigo con errores de ejecución
3    throw 505;
4  }
5  catch (int codigo_error) {
6    cout << "Error: " << codigo_error;
7  }
```